

Lecture 6: Spatial Heterogeneity and Calibration

第六讲：空间异质性与校准

Computational Methods for Heterogeneous-Agent Macro

Jeffrey Sun

May 18, 2026

University of Toronto

Course Overview

Where we are 我们走到哪里了

- L1–L4: build the household block; solve Aiyagari SS.
- L5: meet HouseholdStages.jl; transition dynamics.
- **Today (L6):** a new dimension (space) and a new use of the model (matching it to data).

- L1–L4: 搭家庭块; 求 Aiyagari 稳态。
- L5: 见 HouseholdStages.jl、过渡动态。
- **今天 (L6):** 一个新维度 (空间)、一个新用法 (与数据匹配)。

Today's roadmap

今日路线图

1. A spatial Aiyagari model.
2. MigrationStage: Gumbel discrete choice.
3. Solving the steady state.
4. Calibration: matching population shares.
5. Extensions: goods, trade, factor mobility.
6. Wrap.

1. 空间 Aiyagari 模型。
2. MigrationStage: Gumbel 离散选择。
3. 解稳态。
4. 校准：匹配人口份额。
5. 拓展：商品、贸易、要素流动性。
6. 收尾。

Two new ideas today

今天的两个新想法

Idea 1. Add a new *discrete* dimension to the household state: **location**. A new stage — MigrationStage — runs the within-period location choice.

Idea 2. Use the household block as a *calibration object*: pick model parameters so the implied population shares match the data.

想法一。给家庭状态加一个离散维度：**地点**。新 Stage MigrationStage 负责期内地点选择。

想法二。把家庭块当作校准对象：选择模型参数，让模型隐含的人口份额匹配数据。

A Spatial Aiyagari

Setting: three locations

设定：三个地点

- Three locations loc1, loc2, loc3.
- Each produces **the same single good**: a costlessly-tradable, perfectly-substitutable numeraire.
- Each has its own **productivity** A_j : $A_1 > A_2 > A_3$.
- Local production: $Y_j = A_j K_j^\alpha L_j^{1-\alpha}$.

- 三个地点 loc1、loc2、loc3。
- 每地生产**同一种商品**：无成本可贸易、完全可替代的计价品。
- 每地有自己的**生产率** A_j ： $A_1 > A_2 > A_3$ 。
- 本地生产： $Y_j = A_j K_j^\alpha L_j^{1-\alpha}$ 。

Households

家庭

- Each household has one unit of inelastic labor.
- **State** this period: (b, j) — wealth + current location.
- **No** idiosyncratic income shocks.
Within j , every household earns w_j .
- All heterogeneity is over (b, j) .
- 每户家庭非弹性供给一单位劳动。
- 当期**状态**: (b, j) ——财富 + 当前地点。
- **没有**个体收入冲击；在地点 j 内每户工资都是 w_j 。
- 异质性全在 (b, j) 上。

One bond market

一个统一债券市场

- **One asset**, traded freely across the three locations and with the rest of the world.
- Capital flows freely $\Rightarrow$ **MPK equalized**: $r + \delta = \alpha A_j (L_j / K_j)^{1-\alpha}$.
- Wages adjust:
$$w_j = (1 - \alpha) A_j \left(\frac{\alpha A_j}{r + \delta} \right)^{\alpha/(1-\alpha)}.$$

- **单一资产**，在三地之间以及与世界自由交易。
- 资本自由流动 $\Rightarrow$ **MPK 相等**:
$$r + \delta = \alpha A_j (L_j / K_j)^{1-\alpha}。$$
- 工资随之调整：
$$w_j = (1 - \alpha) A_j \left(\frac{\alpha A_j}{r + \delta} \right)^{\alpha/(1-\alpha)}。$$

Where does r come from?

r 从哪儿来?

Small open economy. The country is *small* in the world bond market $\Rightarrow r$ is taken as given (exogenous, $r = 0.03$).

Why exogenous today? Without idiosyncratic income shocks, household saving is weak; endogenous r would sit at the saving boundary $1/\beta - 1$. Exogenous r lets us focus on **migration + calibration** without bond-market fragility.

Trade-off: the asset market is degenerate; everything else gets a clean treatment.

小型开放经济。 本国在世界债券市场上规模小 $\Rightarrow r$ 视为外生给定 ($r = 0.03$)。

为什么今天外生? 没有个体收入冲击，家庭储蓄不强；内生 r 会顶在储蓄边界 $1/\beta - 1$ 。外生 r 让我们专心于**迁移 + 校准**，不被债券市场的数值不稳干扰。

取舍：资产市场退化，但其余部分干净。

Within-period chain

期内链

1. **Migration.** Draw a Gumbel taste shock over destinations. Pick j' , pay cost $C[j, j']$.
2. **Wealth update.** At destination j' :
 $b' \leftarrow (1 + r)b + w_{j'}$.
3. **Consumption / savings.** Choose next-period wealth.

The chain (compact form):

Migr. $\circ_s$ WealthCh. $\circ_s$ ConsSav.

1. **迁移。**在目的地之间抽 Gumbel 偏好冲击；选目的地 j' ，付成本 $C[j, j']$ 。
 2. **财富更新。**在目的地 j' ：
 $b' \leftarrow (1 + r)b + w_{j'}$ 。
 3. **消费 / 储蓄。**选下期财富。
- 链（紧凑写法）：

Migr. $\circ_s$ WealthCh. $\circ_s$ ConsSav.

What we want to know

要求的对象

Steady-state objects:

- Population shares s_j across locations.
- Wealth distribution $\Lambda(b, j)$ (joint).
- Aggregate $K_{\text{total}} = \sum_b b \Lambda(b, \cdot)$.
- Value function $V(b, j)$.

Prices (r, w_1, w_2, w_3) are pinned down before the household solve.

稳态变量:

- 各地人口份额 s_j 。
- 财富分布 $\Lambda(b, j)$ (联合)。
- 总财富 $K_{\text{total}} = \sum_b b \Lambda(b, \cdot)$ 。
- 价值函数 $V(b, j)$ 。

价格 (r, w_1, w_2, w_3) 在求解家庭前就已经定下。

MigrationStage

The Gumbel choice problem

Gumbel 选择问题

At origin i , a household sees the value of being at each destination j :

$$\tilde{V}_j = -C[i, j] + V_{\text{end}}(j, b),$$

plus an idiosyncratic **Gumbel taste shock** $\varepsilon_j \sim \text{EV1}$ (i.i.d. over j).

The household picks

$j^* = \arg \max_j (\tilde{V}_j + \varepsilon \cdot \varepsilon_j)$, where ε is the shock **scale**.

在原地 i , 家庭看到去每个目的地 j 的价值:

$$\tilde{V}_j = -C[i, j] + V_{\text{end}}(j, b),$$

再加上个体 **Gumbel 偏好冲击** $\varepsilon_j \sim \text{EV1}$ (关于 j 独立同分布)。

家庭选 $j^* = \arg \max_j (\tilde{V}_j + \varepsilon \cdot \varepsilon_j)$, 其中 ε 是冲击的**尺度**。

The log-sum-exp formula

对数-求和-指数公式

With $\varepsilon_j \sim \text{EV1}$, two classical facts:

(a) Choice probabilities are logit:

$$\Pr(j \mid i, b) = \frac{\exp(\tilde{V}_j/\varepsilon)}{\sum_k \exp(\tilde{V}_k/\varepsilon)}.$$

(b) Expected value is the inclusive value:

$$\mathbb{E} \left[\max_j (\tilde{V}_j + \varepsilon \cdot \varepsilon_j) \right] = \varepsilon \log \sum_k \exp(\tilde{V}_k/\varepsilon).$$

若 $\varepsilon_j \sim \text{EV1}$, 有两条经典结论:

(a) 选择概率是 logit:

$$\Pr(j \mid i, b) = \frac{\exp(\tilde{V}_j/\varepsilon)}{\sum_k \exp(\tilde{V}_k/\varepsilon)}.$$

(b) 期望值是 inclusive value:

$$\mathbb{E} \left[\max_j (\tilde{V}_j + \varepsilon \cdot \varepsilon_j) \right] = \varepsilon \log \sum_k \exp(\tilde{V}_k/\varepsilon).$$

Why log-sum-exp matters

为什么 log-sum-exp 重要

- $\max$ is non-smooth, non-differentiable at ties.
 - $\varepsilon \log \sum \exp(\cdot/\varepsilon)$ is **smooth** in its arguments.
 - As $\varepsilon \rightarrow 0$: recovers $\max$ (the argmax wins).
 - As $\varepsilon \rightarrow \infty$: all choices equally likely (random allocation).
 - ε is a **smoothing** / preference-noise parameter.
- $\max$ 不光滑、并列时不可微。
 - $\varepsilon \log \sum \exp(\cdot/\varepsilon)$ 在其参数上**光滑**。
 - $\varepsilon \rightarrow 0$: 退化为 $\max$ (argmax 取胜)。
 - $\varepsilon \rightarrow \infty$: 所有选择等概率 (随机分配)。
 - ε 是**平滑** / 偏好噪声参数。

The MigrationStage kernel

MigrationStage 核

Backward pass (computes V_{pre}):

$$V_{\text{pre}}(b, i) = \varepsilon \log \sum_j \exp \frac{-C[i, j] + V_{\text{end}}(b, j)}{\varepsilon}.$$

Forward pass (pushes Λ):

$$\Lambda_{\text{post}}(b, j) = \sum_i \Pr(j \mid i, b) \Lambda_{\text{pre}}(b, i).$$

Same kernel as the static logit, threaded through the chain by V/Λ duality.

反向 (算 V_{pre}):

$$V_{\text{pre}}(b, i) = \varepsilon \log \sum_j \exp \frac{-C[i, j] + V_{\text{end}}(b, j)}{\varepsilon}.$$

前向 (推 Λ):

$$\Lambda_{\text{post}}(b, j) = \sum_i \Pr(j \mid i, b) \Lambda_{\text{pre}}(b, i).$$

核心和静态 logit 一致，通过 V/Λ 对偶嵌入链中。

The API

API

```
migration = MigrationStage(layout;  
    location_axis = :location,  
    migration_cost = C,      # (n_loc, n_loc) origin → destination  
    ε              = 1.5,    # Gumbel scale  
)
```

- No user closure needed for the cost — pass the matrix.
- ε accepts a literal or a Param (for sweeps / AD).
- Stage allocates the $(\text{cells}, n_{\text{loc}})$ probability tensor itself.
- 成本不需要闭包——直接传矩阵。
- ε 可为字面量或 Param（扫参/自动微分时用）。
- Stage 自己开 $(\text{cells}, n_{\text{loc}})$ 概率张量。

Preference shifters α_j

偏好平移 α_j

We want a per-destination utility add-on α_j :

$$\tilde{V}_j = -C[i, j] + \alpha_j + V_{\text{end}}(j, b).$$

Trick: α_j is the same as a column subtraction from the cost matrix:

$$C_{\text{eff}}[i, j] = C_{\text{base}}[i, j] - \alpha_j.$$

No API change needed — just rebuild (or mutate) the cost.

我们想给每个目的地加一个效用增量 α_j :

$$\tilde{V}_j = -C[i, j] + \alpha_j + V_{\text{end}}(j, b).$$

技巧: 给目的地加 α_j 与把成本矩阵的对应列减去 α_j 等价:

$$C_{\text{eff}}[i, j] = C_{\text{base}}[i, j] - \alpha_j.$$

无需修改 API —— 重建（或就地修改）成本即可。

The household block in code

家庭块代码

```
layout = StateLayout(  
  StateAxis(:wealth, continuous_grid(0, 25; length=250)),  
  StateAxis(:location, categorical([:loc1, :loc2, :loc3])))  
migration = MigrationStage(layout; location_axis = :location,  
  migration_cost = effective_cost(p),  $\varepsilon$  = p. $\varepsilon$ _logit)  
receipt = WealthChangeStage(layout; wealth_axis = :wealth,  
  wealth_post = (cell; env) -> (1 + env.r) * cell.wealth +  
    (cell.location == :loc1 ? env.w1 :  
      cell.location == :loc2 ? env.w2 : env.w3))  
savings = ConsumptionSavingsStage(layout;  $\beta$ =p. $\beta$ , wealth_axis=:wealth,  
  utility = (cell, c; env) -> u_crra(c, Val(p. $\sigma$ )))  
  
hh = lift_moments(migration  $\circ_s$  receipt  $\circ_s$  savings; K_total=..., L1=..., ...)
```

Steady State

No outer loop today

今天没有外层循环

- L4: outer loop on $\bar{K}$ (Aiyagari).
- L5: outer loop on $\{K_t\}$ (transitions).
- **L6: no outer loop.** r is exogenous; $w_j(r, A_j)$ closed-form.
- One inner solve $\rightarrow$ done.

What's left:

`solve_steady_state_given_env!(hh,
env, buffers).`

- L4: 外层迭代 $\bar{K}$ (Aiyagari)。
- L5: 外层迭代路径 $\{K_t\}$ (过渡)。
- **L6: 没有外层循环。** r 外生; $w_j(r, A_j)$ 有闭式。
- 一次内层求解 $\rightarrow$ 完事。

留下来的是

`solve_steady_state_given_env!(hh,
env, buffers)。`

Inner solve

内层求解

```
function solve_household(hh, buffers, p; V_init = nothing,  $\Lambda$ _init = nothing)
    env = (; r = p.r,
             w1 = spatial_wage(p.r, p.A[1], p),
             w2 = spatial_wage(p.r, p.A[2], p),
             w3 = spatial_wage(p.r, p.A[3], p))
    res = solve_steady_state_given_env!(hh, env, buffers;
                                         V_init = V_init,  $\Lambda$ _init =  $\Lambda$ _init)

    m = compute_moments(hh, env)
    shares = [m.L1, m.L2, m.L3] ./ (m.L1 + m.L2 + m.L3)
    return (; res..., env, moments = m, shares)
end
```

V backward to fixed point at env, then Λ forward to stationarity. Then read moments.

V 在 env 下反向收敛, Λ 前向收敛到平稳。然后读取矩。

Uncalibrated baseline ($\alpha = 0$)

未校准基准 ($\alpha = 0$)

With $r = 0.03$ and $A = (1.20, 1.00, 0.85)$:

- Wages:
 $(w_1, w_2, w_3) = (1.66, 1.25, 0.97)$.
- Shares: $(0.39, 0.37, 0.25)$.
- $K_{\text{total}} = 0.75$.

loc1's high productivity makes it the most attractive destination; loc3's low wage drives population out.

取 $r = 0.03$, $A = (1.20, 1.00, 0.85)$:

- 工资:
 $(w_1, w_2, w_3) = (1.66, 1.25, 0.97)$ 。
- 份额: $(0.39, 0.37, 0.25)$ 。
- $K_{\text{total}} = 0.75$ 。

loc1 高生产率最具吸引力; loc3 工资低则人口流出。

Reading V across locations

解读跨地点的 V

At fixed wealth b :

$$V(b, j) \geq V(b, j')$$

purely from wage differences and the cost structure $C[i, j]$.

Migration choice comes from comparing $V_{\text{end}}(j, b')$ at the wealth b' implied by moving to j .

(Implementation: the MigrationStage kernel populates choice_prob, a $(N_w, 3, 3)$ tensor.)

财富 b 给定时:

$$V(b, j) \geq V(b, j')$$

完全由工资差和成本结构 $C[i, j]$ 决定。

迁移选择比较的是搬到 j 后到达财富 b' 时的 $V_{\text{end}}(j, b')$ 。

(实现: MigrationStage 核填充 choice_prob, 一个 $(N_w, 3, 3)$ 张量。)

Calibration

Calibration: model meets data

校准：模型与数据相遇

Familiar example (L4): pick β so that the model's K/Y matches the data K/Y . One parameter, one moment.

Today: pick **preference shifters**

$\alpha = (\alpha_1, \alpha_2, \alpha_3)$ so that the model's **population shares** match the data shares.

Three parameters, three moments.

(Normalization $\alpha_1 = 0$ leaves two free; shares sum to 1 leaves two binding.)

熟悉的例子 (L4): 选 β 让模型的 K/Y 与数据匹配。一个参数、一个矩。

今天: 选**偏好平移** $\alpha = (\alpha_1, \alpha_2, \alpha_3)$ 让模型的人口份额匹配数据。

三个参数、三个矩。(标准化 $\alpha_1 = 0$ 剩两个自由; 份额和为 1 剩两个有效约束。)

The data we want to match

要匹配的数据

Synthetic target:

$$s^{\text{data}} = (0.30, 0.30, 0.40).$$

The data say loc3 houses 40% of households — even though its productivity (and wage) is *lowest*.

Something keeps people in loc3 that the $\alpha = 0$ model does not see: an "amenity," a non-pecuniary preference for loc3. The shifter α_3 will quantify it.

合成目标:

$$s^{\text{data}} = (0.30, 0.30, 0.40).$$

数据显示 loc3 住了 40% 的家庭——尽管它的生产率（和工资）最低。

有什么因素把人留在 loc3，是 $\alpha = 0$ 模型看不到的：一种“便利度”或对 loc3 的非货币偏好。 α_3 将给它一个量值。

The static intuition

静态直觉

In a **static** multinomial logit with deterministic-part utilities δ_j :

$$s_j = \frac{\exp(\delta_j/\varepsilon)}{\sum_k \exp(\delta_k/\varepsilon)}.$$

Take log differences:

$$\log s_j - \log s_{j'} = \frac{\delta_j - \delta_{j'}}{\varepsilon}.$$

If δ_j contains α_j additively, we recover $\alpha_j - \alpha_{j'}$ directly from observed log-share differences.

静态多项 logit 中, 效用确定部分为 δ_j :

$$s_j = \frac{\exp(\delta_j/\varepsilon)}{\sum_k \exp(\delta_k/\varepsilon)}.$$

取 log 差:

$$\log s_j - \log s_{j'} = \frac{\delta_j - \delta_{j'}}{\varepsilon}.$$

如果 δ_j 加性包含 α_j , 那么直接由 log-份额差就能恢复 $\alpha_j - \alpha_{j'}$.

Berry's contraction (1994)

Berry 收缩 (1994)

In our dynamic model, δ_j depends on α through V . The static formula is no longer an exact inversion.

Berry's idea: use it as a *contraction step*:

$$\alpha_j^{(k+1)} = \alpha_j^{(k)} + \varepsilon \cdot (\log s_j^{\text{data}} - \log s_j^{\text{model},(k)}).$$

In the static case Berry proves this is a contraction. In our dynamic case it's still a useful fixed-point iteration if log-shares are monotone in α .

动态模型中， δ_j 通过 V 依赖于 α 。静态公式不再是精确的反演。

Berry 的想法：把它当作收缩步：

$$\alpha_j^{(k+1)} = \alpha_j^{(k)} + \varepsilon \cdot (\log s_j^{\text{data}} - \log s_j^{\text{model},(k)}).$$

静态情形 Berry 证明这是一个收缩。动态情形下，只要 log-份额与 α 单调，仍是有效的不动点迭代。

Why this is "indirect"

为什么叫"间接"

Direct inference: read off the parameter from the data. We can't — we don't observe α_j in any data set.

Indirect inference (broad sense): use the *model* as a translator: observe shares (which we have) and back out α (which we don't) by matching.

Note. Strict Gourieroux–Monfort–Renault (1993) uses an auxiliary model. Macro talk uses the term more loosely; we follow the loose usage.

直接推断: 从数据直接读出参数。这里做不到。

间接推断 (宽义): 把模型当翻译器: 观察份额, 匹配反推 α 。

说明。 严格 Gourieroux–Monfort–Renault (1993) 用辅助模型; 宏观常用宽义, 本讲采用宽义。

The calibration loop

校准循环

1. Initialize $\alpha^{(0)} = 0$.
2. **Solve the household** at $\alpha^{(k)}$. Get $s^{\text{model},(k)}$.
3. Compute gap
 $g_j = \log s_j^{\text{data}} - \log s_j^{\text{model},(k)}$.
4. If $\|g\|_\infty < \text{tol}$, **stop**.
5. Update $\alpha^{(k+1)} = \alpha^{(k)} + \varepsilon \cdot g$,
normalize $\alpha_1 = 0$.
6. Repeat.

1. 初始化 $\alpha^{(0)} = 0$ 。
2. 在 $\alpha^{(k)}$ 上**求解家庭块**。得到 $s^{\text{model},(k)}$ 。
3. 算差 $g_j = \log s_j^{\text{data}} - \log s_j^{\text{model},(k)}$ 。
4. 若 $\|g\|_\infty < \text{tol}$ ，**停止**。
5. 更新 $\alpha^{(k+1)} = \alpha^{(k)} + \varepsilon \cdot g$ ，归一化
 $\alpha_1 = 0$ 。
6. 重复。

In code

代码

```
function calibrate_shifters!(hh, buffers, p, s_data; tol=5e-3, maxiter=30)
     $\alpha$  = copy(p. $\alpha$ _pref); V,  $\Lambda$  = nothing, nothing
    for k in 1:maxiter
        set_shifters!(hh, p,  $\alpha$ )
        out = solve_household(hh, buffers, p; V_init=V,  $\Lambda$ _init= $\Lambda$ )
        V,  $\Lambda$  = out.V, out. $\Lambda$ 
        gap = log.(s_data) .- log.(max.(out.shares, 1e-8))
        maximum(abs.(gap)) < tol && return  $\alpha$ 
         $\alpha$  .+= p. $\varepsilon$ _logit .* gap;     $\alpha$  .-=  $\alpha$ [1]    # Berry step + normalize
    end
end
```

Note. hh.stages[1].migration_cost is mutated in place each iteration — no rebuilding.

注：每轮原地修改成本矩阵，不重建链。

Running it 运行

Iteration log (selected):

- Iter 1: shares (0.387, 0.365, 0.248);
 $\alpha = 0$.
- Iter 2: shares (0.247, 0.279, 0.474);
 $\alpha = (0, 0.09, 1.10)$.
- Iter 4: shares (0.285, 0.297, 0.418);
 $\alpha = (0, 0.03, 0.83)$.
- Iter 8: shares (0.299, 0.300, 0.401);
 $\alpha = (0, -0.013, 0.747)$. **Done.**

Converges in 8 iterations. Geometric convergence in the log-share gap.

迭代记录 (节选):

- 第 1 轮: 份额 (0.387, 0.365, 0.248);
 $\alpha = 0$ 。
- 第 2 轮: 份额 (0.247, 0.279, 0.474);
 $\alpha = (0, 0.09, 1.10)$ 。
- 第 4 轮: 份额 (0.285, 0.297, 0.418);
 $\alpha = (0, 0.03, 0.83)$ 。
- 第 8 轮: 份额 (0.299, 0.300, 0.401);
 $\alpha = (0, -0.013, 0.747)$ 。**完成。**

8 轮收敛。log-份额差几何级下降。

Reading the calibrated α

解读校准的 α

- $\alpha_1 = 0$ (normalized).
- $\alpha_2 \approx -0.013$: loc2's 30% share is consistent with the wage-only model.
- $\alpha_3 \approx 0.747$: loc3 has ≈ 0.75 utility per period of unobserved amenity.

$\alpha_3 \approx 0.75$ is the kind of inference indirect identification gives us: a value for a quantity we cannot directly see.

- $\alpha_1 = 0$ (标准化)。
- $\alpha_2 \approx -0.013$: loc2 的 30% 与工资模型一致。
- $\alpha_3 \approx 0.747$: loc3 每期约 0.75 个单位的未观测便利度。

$\alpha_3 \approx 0.75$ ——间接识别给出一个无法直接观察的量的数值。

What if data shares were different? 数据份额不同会怎样？

Comparative static of identification.

- If s_3^{data} rose from 0.40 to 0.45, α_3 would rise to keep the model honest.
- If s_3^{data} fell to 0.20 (less than the wage-only prediction), α_3 would go *negative* — a *disamenity* that the wage model fails to capture.

The inferred α **tracks the data**, not the model's preferences. That's what makes it a useful empirical object.

识别的比较静态。

- 若 s_3^{data} 由 0.40 升到 0.45, α_3 也升以保持模型与数据一致。
- 若 s_3^{data} 降到 0.20 (低于工资单独的预测), α_3 变成负的 —— 工资模型看不到的负便利度。

推断出的 α **跟随数据走**, 而不是模型偏好。这正是它有用之处。

Extensions

What our model assumed

模型的假设

We assumed:

1. **One good**, perfectly substitutable across locations.
2. **Costless trade** in that good.
3. **Free capital** flow (one r).
4. **Free labor** flow (subject to migration cost).

These are strong. Spatial economics tracks what changes when each is relaxed.

No code today — just the headline.

本讲假设：

1. **一种商品**，地点间完全可替代。
2. 该商品**无成本贸易**。
3. 资本**自由流动**（单一 r ）。
4. 劳动**自由流动**（受迁移成本约束）。

每条都很强。空间经济学研究的是放松每条会变什么。

今天不写代码——只讲方向。

Variation 1: differentiated goods (Armington)

变体 1: 差异化商品 (Armington)

Each location's good is a *distinct variety*; consumers love variety (CES aggregator).

- Trade flows from variety demand.
- Local prices p_j differ; real wages depend on the local price index.
- Migration responds to *real* wages.

This is the **Armington (1969)** structure underlying modern trade / spatial economics (Eaton–Kortum, Allen–Arkolakis).

每地商品是独立变种；消费者偏好多样性 (CES 聚合)。

- 变种需求产生贸易流。
- 各地价格 p_j 不同；实际工资取决于本地价格指数。
- 迁移响应实际工资。

这是现代贸易 / 空间经济学的标准 **Armington (1969)** 结构 (Eaton–Kortum, Allen–Arkolakis)。

Variation 2: trade costs

变体 2：贸易成本

Goods don't cross borders for free: **iceberg cost** $\tau_{ij} \geq 1$ (you must ship τ_{ij} units for 1 to arrive).

- Local prices depend on distance.
- Some goods become non-tradable ($\tau \rightarrow \infty$): services, housing.
- Households consume a mix of tradables and non-tradables.
- Migration costs and trade costs interact (closer regions tend to share both).

商品过境并非无代价：**冰山成本** $\tau_{ij} \geq 1$ (运出 τ_{ij} 单位才到 1 单位)。

- 各地价格随距离变化。
- 部分商品变为不可贸易 ($\tau \rightarrow \infty$): 服务、住房。
- 家庭消费可贸易品 + 不可贸易品的混合。
- 迁移成本与贸易成本相互影响 (邻近地区往往两者都低)。

Variation 3: restricted factor mobility

变体 3：要素流动受限

- **Capital is local** (e.g. housing). Local capital scarcity raises local rental rates and house prices.
- **Migration cost is sunk and large.** Workers don't equalize wages quickly; spatial wage differentials persist.
- **Skill heterogeneity.** High- and low-skill workers face different migration choices — as in modern urban literature (Diamond 2016, Moretti 2011).
- **资本本地化**（如住房）。本地资本稀缺推高租金和房价。
- **迁移成本大且沉没。** 工人不会迅速使工资均等；空间工资差长期存在。
- **技能异质性。** 高低技能工人面临不同的迁移决策——见现代城市文献 (Diamond 2016, Moretti 2011)。

Putting the pieces back 把组件拼回来

Each variation slots into the chain as a *stage* or a *block-level extension*:

- Armington / non-tradables $\Rightarrow$ *change* the env (add p_j , real wages).
- Local housing $\Rightarrow$ *add an axis* (housing) and a stage.
- Skill heterogeneity $\Rightarrow$ *add an axis* (skill) and stage closures keyed on it.

每个变体都能作为 *stage* 或 块级拓展 嵌入链中:

- Armington / 不可贸易 $\Rightarrow$ 改 env (增加 p_j 、实际工资)。
- 本地住房 $\Rightarrow$ 加一维 (住房) 和一个 stage。
- 技能异质性 $\Rightarrow$ 加一维 (技能), stage 闭包按其分支。